

To handle creating new tasks, you need to read the request body and validate the input before updating your mock list. Below is the updated code for both lanes.

● Node.js Lane (Express)

Express requires a middleware line (`app.use(express.json())`) to read JSON bodies. Update your `server.js` file:

javascript

```
const express = require('express');
const app = express();
const PORT = 3000;

// Middleware to parse JSON request bodies
app.use(express.json());

let tasks = [
  { id: 1, title: "Buy groceries", done: false },
  { id: 2, title: "Clean the room", done: true },
  { id: 3, title: "Study API design", done: false }
];

app.get('/', (req, res) => {
  res.json({ name: "Task API", version: "1.0", endpoints: ["/tasks"] });
});

app.get('/health', (req, res) => {
  res.json({ status: "ok" });
});

app.get('/tasks', (req, res) => {
  res.json(tasks);
});

app.get('/tasks/:id', (req, res) => {
  const taskId = parseInt(req.params.id);
  const task = tasks.find(t => t.id === taskId);
  if (!task) {
    return res.status(404).json({ error: `Task ${taskId} not found` });
  }
  res.json(task);
});
```

```
// POST /tasks - Create a new task
app.post('/tasks', (req, res) => {
  const { title } = req.body;

  // Validation: Check if title is missing or empty
  if (!title || title.trim() === "") {
    return res.status(400).json({ error: "Title is required" });
  }

  // Generate next ID
  const nextId = tasks.length > 0 ? Math.max(...tasks.map(t => t.id)) + 1 : 1;

  const newTask = {
    id: nextId,
    title: title,
    done: false
  };

  tasks.push(newTask);
  res.status(201).json(newTask);
});

app.listen(PORT, () => {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

Use code with caution.

● Python Lane (FastAPI)

FastAPI handles validation using Pydantic models automatically. Update your `main.py` file:

```
python

from fastapi import FastAPI, HTTPException, status
from pydantic import BaseModel, Field

app = FastAPI()

# Pydantic model for input validation
```

```
class TaskCreate(BaseModel):
    title: str = Field(..., min_length=1, description="Title cannot be empty")

tasks = [
    {"id": 1, "title": "Buy groceries", "done": False},
    {"id": 2, "title": "Clean the room", "done": True},
    {"id": 3, "title": "Study API design", "done": False}
]

@app.get("/")
def read_root():
    return {"name": "Task API", "version": "1.0", "endpoints": ["/tasks"]}

@app.get("/health")
def read_health():
    return {"status": "ok"}

@app.get("/tasks")
def read_tasks():
    return tasks

@app.get("/tasks/{id}")
def read_task(id: int):
    task = next((t for t in tasks if t["id"] == id), None)
    if task is None:
        raise HTTPException(status_code=404, detail={"error": f"Task {id} not found"})
    return task

# POST /tasks - Create a new task
@app.post("/tasks", status_code=status.HTTP_201_CREATED)
def create_task(task_input: TaskCreate):
    # Additional check for empty spaces strings
    if not task_input.title.strip():
        raise HTTPException(status_code=400, detail={"error": "Title cannot be empty spac

    # Generate next ID
    next_id = max([t["id"] for t in tasks], default=0) + 1

    new_task = {
        "id": next_id,
        "title": task_input.title,
        "done": False
    }
```

```
tasks.append(new_task)
return new_task
```

Use code with caution.

🏁 Checkpoint Verification

Restart your server and run these commands to test:

- **Create valid task:**

bash

```
curl -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{"
```

Use code with caution.

(Expect `201 Created` and the new task object with `id: 4`)

- **Test validation error:**

bash

```
curl -i -X POST http://localhost:3000/tasks -H "Content-Type: application/json" -d '{}
```

Use code with caution.

(Expect `400 Bad Request` and your error JSON message)

- **Verify list growth:** Run `curl http://localhost:3000/tasks` to check if "Buy milk" is now inside the list.

(Note: Swap port `3000` to `8000` if you use Python)

💾 Git Commit

Run these commands in your terminal to save your progress:

bash

```
git add .  
git commit -m "Stage 3: create with validation"
```

Use code with caution.
